

Technical Manual

Smart Invoice Extractor

Technical Test for Verify

Prepared by

Esteban Marulanda

May 9, 2026

Contents

1	Purpose and Scope	1
1.1	Main Capabilities	1
1.2	Methodological Approach	2
1.3	Assumptions of the Current Implementation	2
1.4	Coding Practices Implemented	3
2	General Structure of the Repository	3
2.1	Root-Level Responsibilities	4
3	Architecture and End-to-End Functioning	6
3.1	Service Topology	6
3.2	Processing Flow	6
4	Installation, Startup, and First Use	7
4.1	Prerequisites	7
4.2	Environment Variables	7
4.3	Docker-Based Startup	8
4.4	First Access to the Web Interface	9
4.5	Creating a Project	9
4.6	Opening a Project	9
4.7	Uploading an Invoice Document	9
4.8	Reviewing the Processed Project	9
4.9	Local Python Installation	10
5	Backend Folder	10
5.1	Files and Responsibilities	11
5.2	Database Layer	12
5.3	Models	12
5.4	Authentication	13
5.5	Project API	14
5.6	Pipeline Orchestration	15
6	OCR Folder	17
6.1	Files and Responsibilities	17
6.2	OCR Configuration	18
6.3	Veryfi Adapter	19
6.4	OCR Cache by Content Hash	19
6.5	OCR API Behavior	20
7	Extractor Folder	21
7.1	Files and Responsibilities	22
7.2	DocumentParser	23
7.3	DynamicFormat	25
7.4	Switch Invoice Configuration	27
8	Validator Folder	29
8.1	Files and Responsibilities	30
8.2	ValidatorEngine	30
8.3	Current Validation Rules	32

9 Frontend Folder	34
9.1 Files and Responsibilities	34
9.2 Session State	34
9.3 PDF Rendering	35
10 Logs Folder and Centralized Logging	35
10.1 Logging Design	35
10.2 Service Log Files	35
11 Tests Folder	35
11.1 Testing Structure	35
11.2 Testing Philosophy	36
11.3 Unit Tests	36
11.4 API-Layer Tests	37
11.5 Fixtures and Reports	37
11.6 Running the Tests	37
12 Dockerization	38
12.1 Docker Compose Services	38
12.2 Volumes	38
12.3 Service Dockerfiles	38
13 Configuration-Driven Generalization	39
13.1 Current Generalization Boundary	39
13.2 Adding a New Extractor Format	39
13.3 Adding a New Validator Format	39
13.4 Recommended Future Improvements	40
14 Operational Notes and Known Constraints	40
15 Summary	40

1 Purpose and Scope

This document is a technical guide for the current implementation of the **Smart Invoice Extractor**, developed as a technical test for Verify. The objective is to describe the codebase in enough detail for a developer, reviewer, or maintainer to understand the application architecture, the responsibilities of each folder, the execution flow, the Docker setup, the installation process, the testing strategy, and the future generalization path.

The application processes invoice PDFs through a service-oriented Python architecture. A user uploads a document through a Streamlit interface; the backend stores the PDF and orchestrates an OCR service, an extractor service, and a validator service. The OCR layer uses Verify to obtain raw text, the extractor converts that text into structured JSON using configurable regular expressions, and the validator checks row-level and document-level consistency rules. The current configured format is **Switch Invoice**.

1.1 Main Capabilities

- User registration and login through the backend API.
- Project creation, retrieval, update, deletion, and PDF storage.
- PDF upload and processing from the Streamlit frontend.
- OCR extraction through a Verify adapter.
- OCR result caching by content hash to avoid repeated calls for identical files.
- Dynamic regex-based extraction of header fields, line items, and SKU values.
- Validation of extracted line-item arithmetic and document totals.
- Side-by-side PDF preview and extracted-table review in the frontend.
- Centralized service logging with rotating log files.
- Unit and API-layer tests with fixtures, temporary databases, mocked external services, and coverage reports.

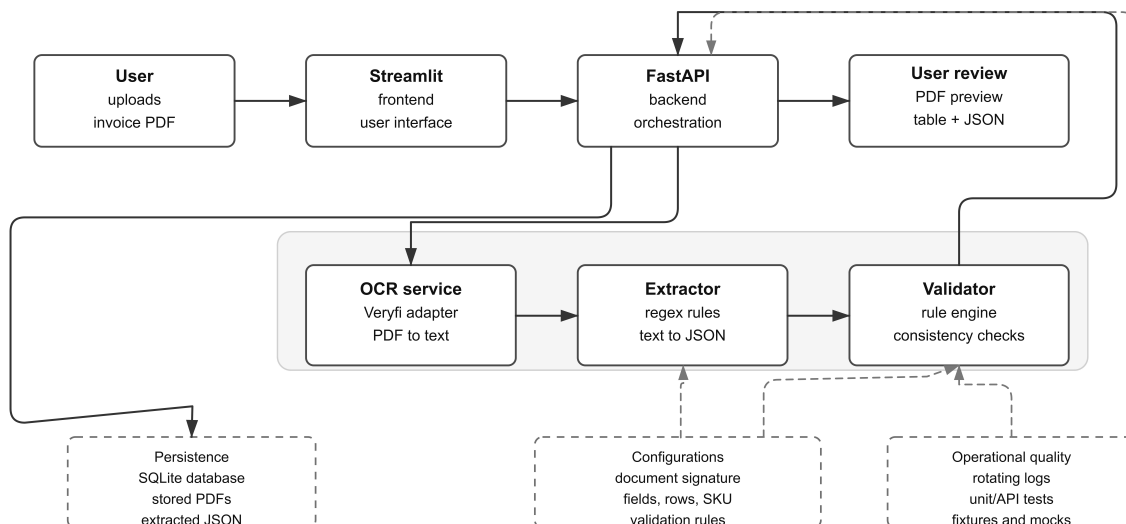


Figure 1: Compact overview of the Smart Invoice Extractor architecture and methodology.

1.2 Methodological Approach

The application is architected as a modular document-processing pipeline, prioritizing decoupling and the Single Responsibility Principle. The workflow is partitioned into five specialized services: the **frontend** manages user state; the **backend** orchestrates the pipeline and persistence; the **OCR service** (via Veryfi adapter) handles PDF-to-text conversion; the **extractor** transforms unstructured text into JSON; and the **validator** ensures data integrity.

This modularity facilitates granular error isolation. By treating the pipeline as a sequence of discrete stages, failure modes—such as OCR timeouts, unknown formats, or parsing exceptions—are localized and easier to debug. This hierarchy is enforced at the orchestration level: while OCR and extraction are mandatory for generating output, validation is a **fail-soft stage**. If the validator fails or is unavailable, the backend still returns the extracted data, appending a fallback object to indicate the status of the quality control check.

The extraction engine follows a **deterministic, configuration-driven approach**. To ensure the core Python logic remains format-agnostic, all document-specific heuristics are externalized into JSON configuration files (e.g., `switch.json`). These files act as declarative templates that parameterize the extraction process through structural anchors and regular expressions.

Under this model, the extractor operates in three distinct phases:

- **Identification:** The system uses a `signature_regex` to verify if the OCR text matches the expected document type.
- **Metadata Extraction:** Header fields are retrieved using a mix of static values (e.g., vendor, currency) and regex-based capture groups for dynamic data like invoice dates and total amounts.
- **Line-Item Parsing:** A parameterized logic uses `start_anchor` and `stop_anchor` to isolate the table area. A `row_pattern` then captures description, quantity, rate, and amount, which are subsequently cast to numeric types after normalizing separators.

A key feature of this methodology is **conditional SKU mapping**. Rather than global inference, SKUs are extracted only when the `description` field matches a specific, configured pattern. This prevents the "hallucination" of identifiers in documents where a reliable SKU pattern is absent. This strategy allows the system to scale; onboarding a new invoice format requires only a new JSON configuration without modifying the core parsing engine.

Finally, the validation layer enforces a strict contract with the extractor. The validator is agnostic to the document's visual origin, processing only the structured JSON. It applies arithmetic consistency rules—verifying the product of quantity and rate against line totals, and cross-referencing the sum of line items with the invoice total within a configured tolerance. As a non-destructive observer, the validator reports on data integrity without altering the underlying extraction results.

1.3 Assumptions of the Current Implementation

The scope of this implementation is defined by a specific set of assumptions designed to focus on the core challenges of the technical test:

- **Template Specificity:** The current configuration is optimized for the **Switch Invoice** format. While the system is extensible, the parser assumes the presence of specific textual anchors (e.g., vendor signatures and table footers) defined in the active configuration.
- **OCR Structural Integrity:** The extraction logic relies on the OCR provider's ability to preserve basic line structure. The methodology assumes that the spatial relationship between fields remains sufficiently consistent for regular-expression-based row parsing.
- **Conditional SKU Mapping:** SKU extraction is not based on global inference but on

explicit pattern matching within the configured source fields. This prevents the generation of "hallucinated" identifiers in cases where the document lacks a standardized numeric SKU.

- **Service Discovery:** The backend assumes a containerized network environment (Docker Compose) where services are reachable via environment-variable-defined URLs.
- **Simplified Security Model:** To prioritize pipeline functionality, the system employs basic session-based authentication within the Streamlit state. It does not implement stateless token-based authorization (e.g., JWT), which would be a prerequisite for a production multi-tenant environment.
- **Data Persistence:** Project data is persisted via SQLite with serialized JSON blobs for extraction results. This approach favors rapid review of full extraction outputs over a highly normalized relational schema for individual line items.

1.4 Coding Practices Implemented

The following engineering standards were maintained to ensure the system is maintainable, scalable, and verifiable:

Separation of Concerns: The project structure enforces strict boundaries. Logic is partitioned into `backend/` (orchestration), `components/ocr/` (text acquisition), `components/extractor/` (structured parsing), `components/validator/` (integrity), `frontend/` (UI), and `tests/` (verification). This prevents leakage between the persistence and processing layers.

Configuration-Driven Logic: By externalizing extraction and validation rules, the system minimizes code changes when onboarding new document formats. This approach shifts the complexity from the Python control flow to declarative JSON definitions.

Service Isolation & HTTP Boundaries: Communication between components occurs exclusively via HTTP requests. This "pluggable" architecture allows individual services—such as the OCR provider—to be replaced or upgraded without impacting the downstream extractor or validator.

Robust Error Handling: The pipeline distinguishes between fatal failures (e.g., OCR unavailability) and non-blocking warnings (e.g., validation mismatches). The OCR service specifically handles edge cases such as empty outputs or initialization timeouts, ensuring the backend receives actionable error telemetry.

Infrastructure as Code & Security: Sensitive credentials (e.g., Veryfi API keys) are managed via environment variables, never hard-coded or logged. Furthermore, the entire stack is containerized using **Docker Compose**, ensuring a reproducible runtime environment and eliminating "it works on my machine" discrepancies during the review process.

Performance Optimization: To minimize latency and API costs, the system implements content-addressable caching for OCR results. By hashing the PDF content (SHA-256), the system avoids redundant external calls for identical files, regardless of their filename.

Test-Driven Reliability: The test suite employs service mocking and temporary in-memory databases. This allows for the validation of the core extraction logic and API contracts in complete isolation from external network dependencies or persistent storage.

2 General Structure of the Repository

The repository is organized around independently runnable services. Each service owns its own Dockerfile and dependency file, while the root `docker-compose.yml` coordinates the full application.

Listing 1: Simplified repository structure

```
project/  
|-- backend/
```

```

| |-- api.py
| |-- database.py
| |-- models.py
| |-- schemas.py
| |-- security.py
| |-- requirements.txt
| |-- Dockerfile
| '-- services.py
|-- components/
| |-- ocr/
| | |-- api.py
| | |-- config.py
| | |-- file_utils.py
| | |-- main.py
| | |-- requirements.txt
| | |-- Dockerfile
| | '-- ocr.py
|-- extractor/
| |-- api.py
| |-- requirements.txt
| |-- Dockerfile
| |-- configs/switch.json
| |-- core/document_parser.py
| |-- formats/base_format.py
| |-- formats/dynamic_format.py
| |-- main.py
| |-- '-- utils/file_handler.py
|-- validator/
| |-- main.py
| |-- requirements.txt
| |-- Dockerfile
| |-- configs/switch.json
| |-- '-- core/rule_engine.py
|-- frontend/
| |-- app.py
| |-- requirements.txt
| |-- Dockerfile
| |-- views/
| |-- utils/
| |-- '-- static/logo.png
|-- logs/
|-- tests/
| |-- api/
| |-- unit/
| |-- fixtures/
| |-- reports/
| |-- conftest.py
| |-- Dockerfile
| |-- '-- requirements.txt
|-- documents/
|-- logger.py
|-- docker-compose.yml
|-- Dockerfile
|-- README.md
|-- TESTING.md
'-- requirements.txt

```

2.1 Root-Level Responsibilities

The root directory contains the orchestration files, shared utilities, documentation, runtime folders, and development artifacts required to run, test, and maintain the application. Table 1 summarizes the role of each relevant root-level item.

Table 1: Root-level files and directories.

Path	Responsibility
<code>backend/</code>	FastAPI orchestration service. It manages authentication, users, projects, PDF persistence, database access, and the execution of the OCR-Extractor-Validator pipeline.
<code>components/</code>	Container for the three processing microservices: OCR, extractor, and validator. Each component has its own API, internal logic, configuration files, Dockerfile, and service-specific dependencies.
<code>frontend/</code>	Streamlit web interface used by the end user. It handles login, project management, PDF upload, extraction execution, PDF preview, validation display, and JSON download.
<code>tests/</code>	Automated testing suite. It contains unit tests, API-layer tests, fixtures, test configuration, and test utilities. This folder verifies the behavior of the parser, validator, OCR utilities, backend services, APIs, logging, and selected frontend helpers.
<code>documents/</code>	Input document folder intended for sample or user-provided PDFs. In this technical test, it is used to store or reference synthetic Switch invoice documents used to exercise the extraction flow.
<code>logs/</code>	Runtime log directory. Each service writes to a dedicated log file through the shared logging utility defined in <code>logger.py</code> . This makes it easier to debug errors in individual services.
<code>.streamlit/</code>	Streamlit configuration directory. It stores application-level frontend settings, such as theme or server configuration, used when running the Streamlit interface.
<code>logger.py</code>	Shared logging factory used across the application. It configures console logging and rotating file logging, prevents duplicated handlers, and centralizes log formatting.
<code>docker-compose.yml</code>	Main multi-container orchestration file. It defines the OCR, extractor, validator, backend, frontend, and test services, including their ports, environment variables, dependencies, and volumes.
<code>Dockerfile</code>	Root-level Dockerfile. It defines a generic OCR-oriented image entry point based on <code>components/ocr/main.py</code> . The service-specific Dockerfiles are located inside the corresponding service folders.
<code>requirements.txt</code>	Root Python dependency file. It contains dependencies needed at the project level. Individual services also define their own dependency files when needed.
<code>README.md</code>	User-facing quick guide. It explains the purpose of the project, how to start it, and how to use the main application workflow.
<code>TESTING.md</code>	Testing guide. It explains how to install test dependencies, run unit tests, run API-layer tests, generate coverage reports, and execute tests through Docker.
<code>.env</code>	Environment variable file. It is expected to contain sensitive configuration such as Veryfi credentials and service settings. This file must not be committed to version control.

Path	Responsibility
<code>.gitignore</code>	Git ignore configuration. It excludes local environments, caches, logs, generated files, credentials, and other artifacts that should not be tracked by Git.
<code>project.zip</code>	Compressed project archive. This is not part of the application runtime; it is a packaging or transfer artifact used to distribute the project during the technical test.
<code>__pycache__</code>	Python bytecode cache generated automatically by the interpreter. It should not be considered part of the source code and should normally be ignored by Git.
<code>.pytest_cache</code>	Pytest cache directory generated when running the test suite. It stores metadata from previous test executions and should not be included in source control.
<code>.venv</code>	Local Python virtual environment. It contains installed dependencies for local development and should not be committed to the repository.

3 Architecture and End-to-End Functioning

3.1 Service Topology

The application uses a microservice-like architecture. The frontend communicates only with the backend. The backend is the orchestration layer and calls the OCR, extractor, and validator services. Each service exposes a FastAPI endpoint, except the frontend, which is a Streamlit application.

Listing 2: Logical request flow

```

Browser user
  -> Streamlit frontend, port 8501
  -> FastAPI backend, port 8000
      -> OCR service, port 8001
      -> Extractor service, port 8002
      -> Validator service, port 8003
  -> SQLite database and uploaded PDF storage

```

3.2 Processing Flow

When the user processes a PDF, the following sequence occurs:

1. The user logs in or registers in the frontend.
2. The user creates or opens a project from the dashboard.
3. The user uploads a PDF in the project workspace.
4. The frontend sends the PDF to `POST /projects/{project_id}/process`.
5. The backend stores the PDF as `/app/data/uploads/project_{id}.pdf`.
6. The backend calls `backend/services.py::run_pipeline()`.
7. `run_pipeline()` calls the OCR service at `/process`.
8. The OCR service computes a SHA-256 hash of the PDF bytes and checks the OCR cache.
9. If the file is not cached, the OCR service writes the PDF to a temporary file and sends it to Veryfi through `VeryfiOCR`.

10. The OCR service returns raw OCR text to the backend.
11. The backend sends that text to the extractor service at `/extract`.
12. The extractor selects the configured document format, extracts header fields and page-level line items, and returns structured JSON.
13. The backend sends the extracted JSON to the validator service at `/validate`.
14. The validator applies configured rules and returns a validation object.
15. The backend attaches validation to the extracted JSON and stores the enriched JSON in the project row.
16. The frontend displays header fields, validation status, page-level line items, and the rendered PDF page.

4 Installation, Startup, and First Use

This section explains how to prepare the environment, start the services, open the web interface, create a project, and upload an invoice document for processing. The recommended execution mode is Docker Compose because the application is composed of multiple services that need to communicate with each other.

4.1 Prerequisites

A developer or reviewer should have:

- Python 3.10 or 3.11 for local development.
- Docker and Docker Compose for containerized execution.
- Veryfi credentials for real OCR execution.
- A terminal capable of running Python and Docker commands.
- A web browser to access the Streamlit interface.

The complete application is designed as a multi-service system. Therefore, running only the frontend is not sufficient for document processing. The OCR, extractor, validator, backend, and frontend services must be available for the complete workflow to operate correctly.

4.2 Environment Variables

The OCR service requires Veryfi credentials. They are loaded from a root-level `.env` file or from the shell environment.

Listing 3: Required OCR environment variables

```
VERYFI_CLIENT_ID=your_client_id
VERYFI_CLIENT_SECRET=your_client_secret
VERYFI_USERNAME=your_username
VERYFI_API_KEY=your_api_key
```

If these credentials are missing, the OCR service cannot initialize the Veryfi client correctly. In that case, the application interface may still start, but real PDF processing through OCR will fail.

The backend requires internal service URLs. In Docker Compose these are injected automatically:

Listing 4: Backend service URLs configured by Docker Compose

```
OCR_URL=http://ocr:8001/process
EXTRACTOR_URL=http://extractor:8002/extract
VALIDATOR_URL=http://validator:8003/validate
```

The frontend uses the backend URL. Inside Docker Compose, this is configured as:

Listing 5: Frontend backend URL inside Docker Compose

```
BACKEND_URL=http://backend:8000
```

These Docker service names are internal to the Docker network. The user should access the application from the browser through the exposed frontend port:

```
http://localhost:8501
```

The logger optionally supports a custom log directory:

```
LOG_DIR=/custom/log/directory
```

The OCR cache optionally supports a custom cache directory:

```
OCR_CACHE_DIR=/custom/ocr/cache
```

4.3 Docker-Based Startup

The recommended way to run the complete application is Docker Compose from the project root:

Listing 6: Starting the full application

```
docker compose up --build
```

This command builds and starts the complete service stack:

Table 2: Application services exposed during Docker execution.

Service	Port	Purpose
Frontend	8501	Streamlit user interface accessed by the reviewer from the browser.
Backend	8000	FastAPI service responsible for authentication, projects, PDF storage, and pipeline orchestration.
OCR	8001	OCR service that integrates with Veryfi and manages OCR cache behavior.
Extractor	8002	Regex-based document extraction service.
Validator	8003	Rule-based validation service for extracted invoice data.

After the containers are running, the user should open the following URL in a browser:

Listing 7: Opening the Streamlit application

```
http://localhost:8501
```

This opens the Streamlit interface of the Smart Invoice Extractor.

4.4 First Access to the Web Interface

When the application is opened for the first time, the user sees the login and registration page. This page is implemented in `frontend/views/login.py` and displayed by the router defined in `frontend/app.py`.

The first screen contains two tabs:

- **Register:** used to create a new local user.
- **Login:** used to enter the application with an existing user.

For a new installation, the reviewer should first create a user from the **Register** tab by entering a username and password. Once the user is created, the reviewer can move to the **Login** tab and authenticate with the same credentials.

4.5 Creating a Project

After login, to create a new project:

1. Open the **Create New Project** tab.
2. Enter a project name.
3. Click the project creation button.

It appears in the **My Projects** tab.

4.6 Opening a Project

To work with a project, the user clicks the **Open** button next to the project name.

4.7 Uploading an Invoice Document

If the selected project has not been processed yet, the project workspace shows the upload mode. To upload and process a document:

1. Click the PDF upload field.
2. Select an invoice PDF document.
3. Click the processing button.

4.8 Reviewing the Processed Project

After the pipeline finishes successfully, the project workspace switches to result mode. The user can then review:

- the validation status;
- the main invoice details;
- the extracted line items;
- the original PDF preview;
- the downloadable extracted JSON;
- the downloadable validation log.

For the configured Switch invoice format, the interface displays fields such as vendor, invoice number, invoice date, due date, bill-to name, vendor address, currency, and total amount.

4.9 Local Python Installation

For local development without Docker, install service dependencies as needed. A minimal setup for root-level Python execution is:

Listing 8: Creating a local Python environment

```
python -m venv .venv
source .venv/bin/activate # Linux/macOS
# .venv\Scripts\activate # Windows PowerShell
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

For a complete local service run, install each service dependency file:

Listing 9: Installing service-specific dependencies

```
python -m pip install -r backend/requirements.txt
python -m pip install -r components/ocr/requirements.txt
python -m pip install -r components/extractor/requirements.txt
python -m pip install -r components/validator/requirements.txt
python -m pip install -r frontend/requirements.txt
```

Then run each service in a separate terminal with the required environment variables:

Listing 10: Running services locally without Docker

```
uvicorn components.ocr.api:app --host 0.0.0.0 --port 8001
uvicorn components.extractor.api:app --host 0.0.0.0 --port 8002
uvicorn components.validator.main:app --host 0.0.0.0 --port 8003
uvicorn backend.api:app --host 0.0.0.0 --port 8000
streamlit run frontend/app.py --server.port 8501
```

For local execution, the service URLs must point to `localhost` instead of Docker service names. For example:

Listing 11: Local backend service URLs

```
OCR_URL=http://localhost:8001/process
EXTRACTOR_URL=http://localhost:8002/extract
VALIDATOR_URL=http://localhost:8003/validate
```

The frontend should also point to the local backend:

Listing 12: Local frontend backend URL

```
BACKEND_URL=http://localhost:8000
```

Once all local services are running, the user can access the application at:

```
http://localhost:8501
```

5 Backend Folder

The `backend/` folder implements the central orchestration service of the application. Its responsibility is not to perform OCR or document parsing directly, but to coordinate the complete workflow between the frontend, the database, the uploaded PDF storage, and the three processing services: OCR, extractor, and validator.

From an architectural point of view, the backend acts as the application control plane. It exposes the HTTP API consumed by the Streamlit frontend, stores users and projects in SQLite, persists uploaded PDF files, and invokes the processing pipeline through asynchronous HTTP calls. The backend is implemented with FastAPI, SQLAlchemy, Pydantic, Passlib, and `httpx`.

A relevant design detail is that authentication is implemented as simple username/password validation. The backend does not currently issue JWT tokens, session cookies, or access tokens. The frontend stores the authenticated user returned by the backend in Streamlit session state. Project creation and project listing depend on a `user_id` query parameter, while project-specific operations use only the project identifier.

5.1 Files and Responsibilities

File	Description
<code>backend/api.py</code>	Defines the FastAPI application and all public backend endpoints. It handles user registration, login, project creation, project listing, project retrieval, project update, project deletion, PDF upload, pipeline execution, and PDF retrieval. It also creates database tables on startup through <code>models.Base.metadata.create_all(bind=engine)</code> .
<code>backend/database.py</code>	Configures the SQLite database, creates the SQLAlchemy engine, defines the session factory, exposes the declarative base, and provides the <code>get_db()</code> dependency used by the FastAPI endpoints. The dependency closes sessions after each request and rolls back if an exception occurs.
<code>backend/models.py</code>	Defines the SQLAlchemy ORM models used by the backend. The current data model contains <code>User</code> and <code>Project</code> . A user can own multiple projects, and each project can store a PDF filename and serialized extracted JSON.
<code>backend/schemas.py</code>	Defines the Pydantic input schemas used by the API: <code>UserCreate</code> , <code>ProjectCreate</code> , and <code>ProjectUpdate</code> . These schemas validate request bodies for authentication and project operations.
<code>backend/security.py</code>	Provides password hashing and verification helpers. Passwords are hashed with <code>bcrypt</code> through <code>Passlib</code> . The verification function also includes a plaintext fallback for legacy records that were not stored as <code>bcrypt</code> hashes.
<code>backend/services.py</code>	Contains the asynchronous pipeline coordinator <code>run_pipeline()</code> . This function reads service URLs from environment variables and calls the OCR, extractor, and validator services in sequence.
<code>backend/Dockerfile</code>	Defines the backend container image. It installs <code>backend/requirements.txt</code> , copies the repository into <code>/app</code> , and starts the API with Uvicorn on port 8000.
<code>backend/requirements.txt</code>	Lists backend-specific dependencies, including FastAPI, Uvicorn, SQLAlchemy, Pydantic, python-multipart, httpx, Passlib, and bcrypt.

The backend also uses the shared `logger.py` module located at the project root. Each backend module obtains a logger with:

```
logger = get_logger(__name__, "backend.log")
```

This means that backend activity is written to the backend log file while also being available through console output. The logged events include database initialization, authentication attempts, project operations, PDF storage, pipeline execution time, and service failures.

5.2 Database Layer

backend/database.py defines the database location using two internal constants:

```
_DB_DIR = "/app/data"
_DB_URL = f"sqlite:///{_DB_DIR}/backend_app.db"
```

The directory /app/data is created automatically before the engine is initialized:

```
os.makedirs(_DB_DIR, exist_ok=True)
```

The backend then creates a SQLite engine with:

```
engine = create_engine(_DB_URL, connect_args={"check_same_thread": False})
```

The check_same_thread=False argument is necessary because FastAPI may handle requests in a way that requires SQLite connections to be usable outside the original thread. The session factory is defined as:

```
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

The get_db() function is the database dependency injected into the FastAPI routes. It yields a SQLAlchemy session, rolls back if an exception is raised, and always closes the session afterward:

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

This pattern ensures that each request receives its own database session and that resources are released after the request finishes. The database tables are created when backend/api.py is imported, using:

```
models.Base.metadata.create_all(bind=engine)
```

In Docker execution, the backend database is stored under /app/data/backend_app.db. When Docker Compose mounts a persistent volume to /app/data, users, projects, extracted JSON, and PDF metadata can survive container restarts.

5.3 Models

backend/models.py defines two ORM models: User and Project.

Model	Field	Meaning
User	id	Integer primary key used internally by the backend.
User	username	Unique username used for login.
User	hashed_password	Password hash generated by hash_password().

User	projects	SQLAlchemy relationship to the projects owned by the user.
Project	id	Integer primary key for the project.
Project	name	Human-readable project name displayed in the dashboard.
Project	owner_id	Foreign key pointing to <code>users.id</code> .
Project	extracted_data	Text column storing the extracted JSON serialized with <code>json.dumps()</code> .
Project	pdf_filename	Original filename of the uploaded PDF.
Project	owner	SQLAlchemy relationship back to the owning user.

The `Project.extracted_data` field is intentionally stored as text rather than as normalized relational tables. During retrieval, `backend/api.py` decodes it with:

```
data = json.loads(project.extracted_data) if project.extracted_data else None
```

This design is simple and appropriate for the current workflow because the extracted document is returned and displayed as a single structured JSON object. However, it also means that the backend does not currently support querying individual invoice fields or line items at the database level.

5.4 Authentication

Authentication is implemented in two layers:

- request handling in `backend/api.py`;
- password hashing and verification in `backend/security.py`.

The password helper functions are:

- `hash_password(password)`: hashes a plaintext password using bcrypt through Passlib.
- `verify_password(plain, hashed)`: verifies a plaintext password against a stored bcrypt hash.

The verification function includes a fallback behavior: if bcrypt verification raises a `ValueError`, the function compares the plaintext values directly. This is explicitly logged as a warning and is intended only for backward compatibility with legacy records.

The backend exposes two authentication endpoints:

Endpoint	Input	Behavior
POST <code>/auth/register</code>	<code>UserCreate</code> body containing <code>username</code> and <code>password</code> .	Checks whether the username already exists. If not, it hashes the password, stores the user, commits the database transaction, and returns a success message. If the username already exists, it returns HTTP 400.
POST <code>/auth/login</code>	<code>UserCreate</code> body containing <code>username</code> and <code>password</code> .	Looks up the user by username and verifies the password. If valid, it returns <code>user_id</code> and <code>username</code> . If invalid, it returns HTTP 400.

The login response is intentionally minimal:


```
{
  "user_id": 1,
  "username": "admin"
}
```

The current backend does not issue an access token. Therefore, authorization is not enforced by a backend middleware. The frontend uses the returned user object to control the visible application state.

5.5 Project API

The project API manages project records and the PDF file associated with each project. The endpoints are defined in `backend/api.py`.

Endpoint	Behavior
POST <code>/projects</code>	Creates a project using a <code>ProjectCreate</code> request body and a <code>user_id</code> query parameter. The new project is stored with <code>owner_id=user_id</code> .
GET <code>/projects</code>	Returns all projects whose <code>owner_id</code> matches the provided <code>user_id</code> query parameter. This is the endpoint used by the dashboard to list the current user's projects.
GET <code>/projects/{project_id}</code>	Returns the selected project's <code>id</code> , <code>name</code> , <code>pdf_filename</code> , and decoded <code>extracted_data</code> . If the project does not exist, it returns HTTP 404.
PUT <code>/projects/{project_id}</code>	Updates the project name using a <code>ProjectUpdate</code> request body. If the project does not exist, it returns HTTP 404.
DELETE <code>/projects/{project_id}</code>	Deletes the project record. If <code>/app/data/uploads/project_{project_id}.pdf</code> exists, it is removed from disk before the database record is deleted. If the project does not exist, it returns HTTP 404.
POST <code>/projects/{project_id}/process</code>	Receives a PDF file as multipart form data. It saves the file, updates the project's <code>pdf_filename</code> , runs the OCR–Extractor–Validator pipeline, stores the extracted JSON, and returns the processed result.
GET <code>/projects/{project_id}/pdf</code>	Returns the stored PDF file as a <code>FileResponse</code> . If the PDF file does not exist, it returns HTTP 404.

The PDF upload directory is defined in `backend/api.py` as:

```
UPLOADS_DIR = "/app/data/uploads"
```

When a PDF is uploaded for processing, it is stored using a deterministic project-based filename:

```
pdf_path = os.path.join(UPLOADS_DIR, f"project_{project_id}.pdf")
```

This means each project has at most one stored PDF at this path. Uploading and processing another PDF for the same project overwrites the previous stored PDF.

The process endpoint follows this sequence:

1. Look up the project by `project_id`.
2. Return HTTP 404 if the project does not exist.

3. Read the uploaded file into memory.
4. Store the original filename in `project.pdf_filename`.
5. Ensure that `UPLOADS_DIR` exists.
6. Write the PDF bytes to `/app/data/uploads/project_{project_id}.pdf`.
7. Commit the PDF filename update.
8. Call `run_pipeline(file.filename, content)`.
9. Serialize the returned JSON with `json.dumps()`.
10. Store it in `project.extracted_data`.
11. Commit the database transaction.
12. Return the processed data to the frontend.

A relevant operational detail is that the PDF file and original filename are saved before the pipeline is executed. If the pipeline later fails, the endpoint returns HTTP 500, but the uploaded PDF file may already exist on disk and the filename may already have been committed to the database.

Another important backend detail is that project-specific endpoints currently identify projects only by `project_id`. Except for `GET /projects`, which filters by `user_id`, the current backend code does not enforce project ownership checks on `GET /projects/{project_id}`, `PUT /projects/{project_id}`, `DELETE /projects/{project_id}`, `POST /projects/{project_id}/process`, or `GET /projects/{project_id}/pdf`. This is acceptable for the current technical-test scope, but it should be addressed before using the backend in a multi-user production environment.

5.6 Pipeline Orchestration

The function `run_pipeline(filename, content)` in `backend/services.py` is the backend's central processing coordinator. It receives the original filename and the PDF bytes, then calls the three internal services in sequence.

The service URLs are read from environment variables:

```
ocr_url = os.getenv("OCR_URL")
extractor_url = os.getenv("EXTRACTOR_URL")
validator_url = os.getenv("VALIDATOR_URL")
```

If any of these variables is missing, the function raises a `RuntimeError`:

```
Required environment variables not set: OCR_URL, EXTRACTOR_URL, VALIDATOR_URL
```

The pipeline uses `httpx.AsyncClient` with a 120-second timeout:

```
async with httpx.AsyncClient(timeout=120.0) as client:
    ...
```

The pipeline sequence is:

Listing 13: Backend pipeline sequence

```
PDF bytes
-> POST OCR_URL with multipart file
-> read response["ocr_text"]
-> POST EXTRACTOR_URL with {"ocr_text": ...}
-> read structured extracted JSON
-> POST VALIDATOR_URL with {"extracted_data": ...}
-> attach validation result to extracted JSON
-> return enriched extracted JSON
```

The first call sends the uploaded PDF to the OCR service as multipart form data:

```
ocr_res = await client.post(
    ocr_url,
    files={"file": (filename, content, "application/pdf")}
)
```

If the OCR service returns any status other than HTTP 200, the backend raises:

```
RuntimeError(f"OCR Service failed: {ocr_res.text}")
```

If OCR succeeds, the backend extracts the OCR text from the response:

```
ocr_text = ocr_res.json().get("ocr_text", "")
```

The second call sends this OCR text to the extractor service:

```
ext_res = await client.post(extractor_url, json={"ocr_text": ocr_text})
```

If the extractor returns any status other than HTTP 200, the backend raises:

```
RuntimeError(f"Extractor Service failed: {ext_res.text}")
```

If extraction succeeds, the backend stores the structured response as `extracted`. This object contains the document format, header fields, pages, line items, and other extraction fields returned by the extractor service.

The third call sends the extracted JSON to the validator service:

```
val_res = await client.post(
    validator_url,
    json={"extracted_data": extracted}
)
```

The validator is handled differently from OCR and extraction. If the validator returns HTTP 200, its JSON response is attached to the extracted object:

```
extracted["validation"] = val_res.json()
```

If the validator does not return HTTP 200, the pipeline does not fail. Instead, the backend attaches a fallback validation object:

```
extracted["validation"] = {
    "is_valid": False,
    "message": f"Validator failed: {val_res.text}",
    "errors": [],
    "details": [],
}
```

This design makes OCR and extraction mandatory, but validation non-blocking. In other words, the backend cannot return a useful processed document without OCR and extraction, but it can still return extracted data if the validator is temporarily unavailable.

The function finally returns the enriched extracted JSON to `process_project()`, where it is serialized and stored in the database.

Overall, the backend implements a clear orchestration pattern:

- FastAPI endpoints receive user and project requests.
- SQLAlchemy persists users, projects, PDF metadata, and extracted JSON.
- The filesystem stores uploaded PDFs by project identifier.
- `run_pipeline()` coordinates the OCR, extractor, and validator services.
- Logging records the main execution steps, failures, and processing time.

6 OCR Folder

The `components/ocr/` folder implements the OCR layer of the application. Its main responsibility is to receive invoice PDF files, obtain OCR text through the Veryfi API, and return that text to the backend pipeline. The folder also includes local command-line utilities for processing a single document outside the web application flow.

In the complete application, the OCR service is not called directly by the Streamlit frontend. Instead, the frontend uploads a PDF to the backend, and the backend forwards the PDF bytes to the OCR service through the internal endpoint `POST /process`. The OCR service then either returns cached OCR text or calls Veryfi to process the document.

A relevant implementation detail is that the OCR API and the OCR CLI share some utility functions but are not identical workflows. The API uses a content-hash cache under `/app/ocr_cache`, while the CLI utility saves OCR output under `components/ocr/output/` using the document filename.

6.1 Files and Responsibilities

File	Description
<code>components/ocr/api.py</code>	Defines the FastAPI OCR service. It exposes <code>POST /process</code> , initializes the module-level <code>ocr_service</code> , reads uploaded PDF bytes, computes a content hash, checks the OCR cache, writes cache misses to a temporary PDF file, calls Veryfi through <code>VeryfiOCR</code> , saves successful OCR results to cache, and removes the temporary file after processing.
<code>components/ocr/config.py</code>	Loads the Veryfi credentials required by the OCR adapter. It searches for a root-level <code>.env</code> file and falls back to environment variables if the file is not found. It validates that all required credentials are present and raises <code>ValueError</code> if any credential is missing.
<code>components/ocr/file_utils.py</code>	Contains file and cache utilities used by both the OCR API and the OCR CLI. It provides SHA-256 hashing, content-hash cache read/write functions, output directory creation, filename-based processed-file detection, and OCR text saving for local CLI execution.
<code>components/ocr/ocr.py</code>	Defines the <code>VeryfiOCR</code> adapter class. This class wraps <code>veryfi.client</code> , sends documents to the Veryfi API, returns the full Veryfi response, and extracts only the <code>ocr_text</code> field when required by the rest of the application.
<code>components/ocr/main.py</code>	Implements the command-line OCR workflow. It loads configuration, initializes <code>VeryfiOCR</code> , checks whether a local document has already been processed, calls OCR if needed, and saves the resulting text under <code>components/ocr/output/</code> .
<code>components/ocr/Dockerfile</code>	Defines the OCR service container. It installs the OCR-specific dependencies from <code>components/ocr/requirements.txt</code> , copies the repository into <code>/app</code> , and starts the FastAPI service with Uvicorn on port 8001.
<code>components/ocr/requirements.txt</code>	Lists the OCR service dependencies: FastAPI, Uvicorn, <code>python-multipart</code> , <code>veryfi</code> , and <code>python-dotenv</code> .

The OCR module also uses the shared project logger through:

```
logger = get_logger(__name__, "ocr.log")
```

This means that OCR configuration, cache behavior, Veryfi calls, temporary file handling, and processing failures are written to the OCR log file.

6.2 OCR Configuration

The configuration logic is implemented in `components/ocr/config.py`. The OCR service requires four Veryfi credentials:

```
VERYFI_CLIENT_ID
VERYFI_CLIENT_SECRET
VERYFI_USERNAME
VERYFI_API_KEY
```

Internally, these environment variables are mapped to the dictionary keys expected by `VeryfiOCR`:

```
_ENV_VAR_MAP = {
    "client_id": "VERYFI_CLIENT_ID",
    "client_secret": "VERYFI_CLIENT_SECRET",
    "username": "VERYFI_USERNAME",
    "api_key": "VERYFI_API_KEY",
}
```

The function `load_configuration()` first resolves the project root by moving three directory levels up from `components/ocr/config.py`. It then searches for a root-level `.env` file:

```
root_dir = os.path.dirname(os.path.dirname(os.path.dirname(__file__)))
env_path = os.path.join(root_dir, ".env")
```

If this file exists, the variables are loaded from that path. If the file does not exist, the function falls back to variables already available in the shell environment:

```
if os.path.exists(env_path):
    load_dotenv(dotenv_path=env_path)
else:
    load_dotenv()
```

After loading the environment, the function builds the configuration dictionary:

```
config = {key: os.getenv(env_var) for key, env_var in _ENV_VAR_MAP.items()}
```

If any value is missing, `load_configuration()` logs the missing logical keys and raises a `ValueError`. The function does not log the actual credential values, only the presence or absence of the expected keys.

On success, the returned dictionary has the following structure:

```
{
    "client_id": "...",
    "client_secret": "...",
    "username": "...",
    "api_key": "..."
}
```

This configuration is consumed by `VeryfiOCR` during service initialization.

6.3 Veryfi Adapter

The file `components/ocr/ocr.py` defines the `VeryfiOCR` class. This class is a thin adapter around `veryfi.Client`. Its purpose is to isolate the rest of the application from the direct Veryfi SDK call.

The constructor receives the dictionary returned by `load_configuration()` and initializes the Veryfi client:

```
self.client = Client(
    config["client_id"],
    config["client_secret"],
    config["username"],
    config["api_key"],
)
```

The class exposes two methods:

- `process_document(file_path)` sends the file located at `file_path` to Veryfi and returns the full response dictionary.
- `extract_ocr_text(file_path)` calls `process_document()` and returns only the `ocr_text` field.

The implementation of `process_document()` delegates directly to:

```
response = self.client.process_document(file_path)
```

If the Veryfi SDK raises an exception, the adapter catches it, logs the failure, and raises a `RuntimeError`. This prevents raw SDK exceptions from propagating directly into the API layer.

The method `extract_ocr_text()` uses:

```
ocr_text = response.get("ocr_text", "")
```

Therefore, if Veryfi returns a response without the `ocr_text` key, the method returns an empty string. The API layer then treats an empty OCR result as an invalid extraction and returns HTTP 400.

6.4 OCR Cache by Content Hash

The current OCR API uses content-based caching to avoid repeated Veryfi calls for identical PDF files. This behavior is implemented in `components/ocr/file_utils.py` and used by `components/ocr/api.py`.

The cache directory is defined at import time with:

```
_CACHE_DIR = os.environ.get("OCR_CACHE_DIR", "/app/ocr_cache")
```

If the environment variable `OCR_CACHE_DIR` is not set, cache files are stored under `/app/ocr_cache`. In Docker execution, this directory can be persisted through a volume.

When the OCR API receives a PDF, it reads the full file into memory and computes a SHA-256 hash of the raw bytes:

```
file_hash = compute_content_hash(content)
```

The hash is independent of the original filename. Therefore, if the same PDF content is uploaded again under another name, the OCR service can still detect it as the same document.

The cache lookup function builds the cache filename as:

```
cache_file = os.path.join(_CACHE_DIR, f"{file_hash}.json")
```

If the cache file exists and can be decoded as JSON, the service returns the cached OCR text without calling Veryfi. If the cache file exists but cannot be read or decoded, the exception is logged and the service treats the situation as a cache miss.

Successful cache payloads are saved with the following fields:

- `file_hash`: SHA-256 hash of the original PDF bytes.
- `original_filename`: filename received in the upload request.
- `processed_at`: UTC timestamp indicating when OCR was performed.
- `character_count`: number of characters in the OCR text.
- `ocr_text`: text extracted from the document.

The cache file is written as UTF-8 JSON with `ensure_ascii=False` and `indent=2`. This makes the cache readable and preserves non-ASCII characters if they appear in OCR output.

It is important to distinguish this API cache from the CLI output folder. The API cache is keyed by content hash and stored under `/app/ocr_cache`. The CLI output is keyed by document filename and stored under:

```
components/ocr/output/<document_name>/<document_name>.txt
```

The CLI output mechanism is useful for local one-file OCR execution, while the API cache is used during the web application pipeline.

6.5 OCR API Behavior

The OCR FastAPI service is defined in `components/ocr/api.py`. When the module is imported, it immediately attempts to initialize the OCR service:

```
try:
    config = load_configuration()
    ocr_service = VeryfiOCR(config)
except Exception:
    ocr_service = None
```

This means that configuration errors are detected during service startup. However, the FastAPI application still exists even if initialization fails. In that case, `ocr_service` remains `None`, and `POST /process` returns HTTP 500 with the message:

```
{
  "detail": "OCR service not initialized."
}
```

The endpoint `POST /process` accepts one uploaded file using FastAPI's `UploadFile` abstraction:

```
async def process_pdf(file: UploadFile = File(...)):
```

The endpoint expects the backend to send the PDF as multipart form data. It does not receive a local file path and does not receive OCR text directly.

The processing sequence is:

1. Log the received filename and content type.
2. Verify that `ocr_service` was initialized.
3. Read the uploaded file bytes into memory.
4. Compute a SHA-256 content hash from the PDF bytes.
5. Look up the OCR cache using the content hash.
6. If a cache entry exists, return `\{"ocr_text":..., "cached":true\}`.
7. If no cache entry exists, write the bytes to a temporary PDF file.
8. Call `ocr_service.extract_ocr_text(temp_pdf_path)`.
9. If the OCR text is empty, return HTTP 400 with `"Not text extracted."`.
10. Save the OCR text to the hash-based cache.

11. Return `\{"ocr_text":..., "cached":false\}`.
12. Remove the temporary PDF file in the `finally` block.

The temporary file is created with:

```
with tempfile.NamedTemporaryFile(delete=False, suffix=".pdf") as temp_pdf:
    temp_pdf.write(content)
    temp_pdf_path = temp_pdf.name
```

The file is created with `delete=False` because the path must remain available after the context manager closes. Veryfi receives the path to this temporary file, not the raw bytes. After OCR processing finishes, the API removes the temporary file:

```
finally:
    if os.path.exists(temp_pdf_path):
        os.remove(temp_pdf_path)
```

The endpoint has three main response categories:

Condition	Response behavior
OCR service not initialized	Returns HTTP 500 with <code>"OCRservicenotinitialized."</code> . This usually means that credentials were missing or the Veryfi client could not be created.
Cache hit	Returns HTTP 200 with <code>ocr_text</code> and <code>cached=true</code> . No Veryfi API call is made.
Cache miss and successful OCR	Calls Veryfi, saves the OCR result to the hash cache, and returns HTTP 200 with <code>ocr_text</code> and <code>cached=false</code> .
Cache miss but empty OCR text	Returns HTTP 400 with <code>"Notextextracted."</code> . The empty result is not saved to cache.
Unexpected processing error	Logs the exception and returns HTTP 500 with <code>"InternalOCRprocessingerror."</code> .

The expected successful response has this structure:

```
{
  "ocr_text": "Extracted document text...",
  "cached": false
}
```

or, when the content-hash cache is used:

```
{
  "ocr_text": "Extracted document text...",
  "cached": true
}
```

This design makes the OCR service deterministic from the backend perspective: for the same PDF bytes, the backend receives the same OCR text, and repeated uploads can avoid unnecessary Veryfi usage.

7 Extractor Folder

The `components/extractor/` folder implements the information extraction layer of the application. This is one of the core components of the project because it transforms unstructured OCR text into structured JSON that can be validated, stored, displayed in the frontend, and downloaded by the user.

The extractor does not perform OCR and does not validate totals. Its input is plain OCR text, normally produced by the OCR service. Its output is a structured dictionary containing the detected document format, extracted header fields, page-level line items, and a global list of all extracted line items.

The methodology used in this project is intentionally configuration-driven. Instead of hard-coding all extraction rules in Python, each supported document type is described by a JSON configuration file inside `components/extractor/configs/`. The Python code provides a generic extraction engine, while the JSON file defines the document-specific rules: how to identify the document, which header fields to extract, where the line-item table starts and stops, how table rows are parsed, and how the SKU is derived.

For the current technical test, the configured document type is `SwitchInvoice`, implemented in:

```
components/extractor/configs/switch.json
```

The extractor is exposed both as an internal FastAPI service and as a local command-line utility. In the complete web application workflow, the backend calls the extractor service through `POST /extract` after receiving OCR text from the OCR service.

7.1 Files and Responsibilities

File	Description
<code>components/extractor/api.py</code>	Defines the FastAPI extractor service. It exposes <code>POST /extract</code> , receives OCR text as JSON, calls the module-level <code>DocumentParser</code> , converts parser errors into HTTP 400 responses, and returns HTTP 500 for unexpected extraction failures.
<code>components/extractor/main.py</code>	Implements the local command-line extraction workflow. It accepts either a path to a text file or raw OCR text, runs the same <code>DocumentParser</code> , and writes successful extraction results to a JSON file under <code>components/extractor/output/</code> or a user-provided output directory.
<code>components/extractor/core/document_parser.py</code>	Implements the orchestration logic of the extractor. It loads all JSON format configurations, creates <code>DynamicFormat</code> objects, splits OCR text into pages, selects the matching document format, extracts header fields from the full text, extracts line items page by page, and builds the final JSON structure.
<code>components/extractor/formats/base_format.py</code>	Defines the abstract interface that any document format parser must implement. It specifies the required methods: <code>format_name</code> , <code>is_match()</code> , <code>extract_header_fields()</code> , and <code>extract_line_items()</code> .
<code>components/extractor/formats/dynamic_format.py</code>	Implements the generic regex-based extraction engine driven by JSON configuration. It handles format detection, static and regex-based header extraction, anchor-delimited table parsing, row parsing, numeric conversion, continuation-line handling, and SKU extraction.

<code>components/extractor/configs/switch.json</code>	Defines the extraction rules for Switch invoices. It contains the format name, signature regex, header-field rules, line-item table anchors, row pattern, output columns, and SKU extraction rule.
<code>components/extractor/utils/file_handler.py</code>	Provides small file utilities for the CLI workflow. It reads UTF-8 text files and writes pretty-printed JSON with <code>ensure_ascii=False</code> , preserving Unicode characters in the output.
<code>components/extractor/Dockerfile</code>	Defines the extractor service container. It installs the dependencies listed in <code>components/extractor/requirements.txt</code> , copies the repository into <code>/app</code> , and starts the FastAPI application with Uvicorn on port 8002.
<code>components/extractor/requirements.txt</code>	Lists the extractor-specific dependencies: FastAPI, Uvicorn, and Pydantic.

The extractor also uses the shared logging utility:

```
logger = get_logger(__name__, "extractor.log")
```

This means that configuration loading, format detection, field extraction, line-item parsing, parser errors, and API failures are written to the extractor log file.

7.2 DocumentParser

The class `DocumentParser`, implemented in `components/extractor/core/document_parser.py`, coordinates the extraction process. It does not contain document-specific extraction rules. Instead, it loads every JSON file located in:

```
components/extractor/configs/
```

For each JSON configuration file, `DocumentParser` creates one `DynamicFormat` object:

```
self.formats.append(DynamicFormat(config_data))
```

This design allows the extractor to support multiple document types without changing the parser orchestration code. Adding support for another invoice format requires adding a new JSON configuration file, provided that the document can be handled by the existing regex-based extraction engine.

If the `configs/` directory does not exist, `DocumentParser` creates it and logs a warning. If an individual JSON file cannot be loaded, the exception is logged and the parser continues loading the remaining configuration files. This prevents a single broken configuration from stopping the extractor service entirely.

The main method is:

```
parse(text: str) -> Dict[str, Any]
```

Its methodology is the following:

1. Split the OCR text into pages using the form-feed character `\T1\textbackslashashx0c`.
2. Remove empty pages and strip leading/trailing whitespace from each page.
3. Use only the first page to identify the document format.
4. Compare the first page against each configured format's `signature_regex`.
5. If no format matches, return an error dictionary.
6. Extract header fields from the full OCR text, not only from the first page.

7. Extract line items independently from each page.
8. Store page-level line items inside the `pages` list.
9. Append all page-level line items into `all_line_items`.
10. Return the final structured JSON object.

The page-splitting logic is:

```
pages = re.split(r"\x0c", text)
return [p.strip() for p in pages if p.strip()]
```

If the document is empty after splitting, the parser returns:

```
{
  "error": "Empty document"
}
```

If no configured format matches the first page, the parser returns an error indicating that the document format has not yet been configured:

```
{
  "error": "El formato de este documento an no ha sido configurado. Por favor agregue el
           archivo JSON de configuracin en la carpeta configs/."
}
```

When the format is detected successfully, the returned structure is:

Listing 14: Extractor output schema

```
{
  "format": "Switch Invoice",
  "header": {
    "vendor_name": "Switch, Ltd.",
    "vendor_address": "Dallas, TX 75267-4592\nPO Box 674592",
    "invoice_date": "09/22/23",
    "due_date": "08/27/24",
    "invoice_number": "1556267",
    "bill_to_name": "Toni Hackel",
    "total_amount": "58,164,641.64",
    "currency": "USD"
  },
  "pages": [
    {
      "page_number": 1,
      "line_items_count": 7,
      "line_items": [
        {
          "sku": "4419693704",
          "description": "Transport | Switch Fiber Pair ...",
          "quantity": 2912.98,
          "rate": 934.09,
          "amount": 2720985.49
        }
      ]
    }
  ],
  "all_line_items": [
    {
      "sku": "4419693704",
      "description": "Transport | Switch Fiber Pair ...",
      "quantity": 2912.98,
      "rate": 934.09,
      "amount": 2720985.49
    }
  ]
}
```

```
]
}
```

The distinction between `pages` and `all_line_items` is important. The frontend uses page-level information to synchronize the PDF preview with the extracted table, while the validator can use `all_line_items` or the page-level line items to validate the complete document.

7.3 DynamicFormat

`DynamicFormat`, implemented in `components/extractor/formats/dynamic_format.py`, is the main extraction engine. It receives a JSON dictionary and converts its rules into runtime extraction behavior.

At initialization, it reads:

- `format_name`: human-readable name of the document type.
- `signature_regex`: regular expression used to identify this document format.
- `header_fields`: rules for extracting header-level information.
- `line_items`: rules for locating and parsing the line-item table.
- `line_items.sku`: optional rule for deriving a SKU from one of the extracted line-item fields.

The class provides the following methods:

Method	Role
<code>is_match(first_page_text)</code>	Detects whether the first page of the OCR text belongs to this format. It returns <code>False</code> if no <code>signature_regex</code> is configured. Otherwise, it applies <code>re.search()</code> to the first page.
<code>extract_header_fields(page_text)</code>	Extracts document-level fields using the configured <code>header_fields</code> . It supports both static fields and regex-based fields. Missing regex matches are returned as <code>None</code> .
<code>extract_line_items(page_text)</code>	Extracts table rows from a single page. It detects the table using <code>start_anchor</code> , stops at <code>stop_anchor</code> , applies <code>row_pattern</code> , maps capture groups to configured columns, converts numeric fields to floats, handles continuation lines, and resolves SKU values.
<code>_extract_sku(item)</code>	Private helper that extracts a SKU from a configured source field, normally <code>description</code> . It returns the matched group as a string or <code>None</code> when no SKU can be resolved.

The extraction methodology inside `extract_line_items()` is the core of the parser. It can be summarized as an anchor-delimited table extraction algorithm:

1. Split the page text into individual lines.
2. Ignore empty lines.
3. Search line by line until the `start_anchor` matches.
4. Once the start anchor is found, set `in_table=True`.
5. For each subsequent line, check whether the `stop_anchor` matches.
6. If the stop anchor matches, finish table parsing for that page.
7. If the row matches `row_pattern`, create a new structured item.
8. Before creating a new item, append the previous pending item to the output list.

9. Map regex capture groups to the configured `columns`.
10. Convert `quantity`, `rate`, and `amount` to floating-point numbers.
11. Insert `sku` as the first key in the item.
12. Resolve the SKU using `_extract_sku()`.
13. If a line does not match `row_pattern` but an item is already open, treat that line as a continuation of the first configured column.
14. Recompute the SKU after appending continuation text.
15. After all lines have been processed, append the final pending item.

The following code fragment captures the essence of the row parsing logic:

```
match = row_pattern.match(line)

if match:
    if current_item:
        line_items.append(current_item)

        current_item = {"sku": None}

    for i, col in enumerate(columns, start=1):
        val = match.group(i).strip()
        if col in ("quantity", "rate", "amount"):
            val = float(val.replace(",", "")) if val else 0.0
            current_item[col] = val

    current_item["sku"] = self._extract_sku(current_item)
```

The parser deliberately inserts `sku` as the first key:

```
current_item = {"sku": None}
```

This is done so that when the data is converted into a table in the frontend, the SKU appears as the first column due to Python dictionary insertion order.

The handling of continuation lines is also important. OCR text can break long descriptions across multiple lines. When a line does not match the row pattern, but a `current_item` already exists, the extractor appends that line to the first configured column:

```
first_col = columns[0]
current_item[first_col] += " " + line
current_item["sku"] = self._extract_sku(current_item)
```

In the current Switch configuration, the first column is `description`. Therefore, continuation lines are appended to the description. After appending the extra text, SKU extraction is evaluated again. This matters because the SKU pattern may only match after the full description has been reconstructed.

The numeric conversion is limited to the configured columns named:

```
quantity
rate
amount
```

For these fields, commas are removed before conversion:

```
float(val.replace(",", ""))
```

This allows values such as 2,912.98 or 2,720,985.49 to be converted into Python floats. Negative values are supported if the configured `row_pattern` allows a leading minus sign.

If no `line_items` rules are defined in a configuration, `extract_line_items()` returns an empty list.

7.4 Switch Invoice Configuration

The current extraction configuration is defined in:

```
components/extractor/configs/switch.json
```

This JSON file contains the extraction rules for the current invoice type used in the technical test.

The document is identified as a Switch invoice using the following signature:

```
"signature_regex": "(?i)switch\\s+Dallas,\\s+TX"
```

The (?i) flag makes the match case-insensitive. The expression searches for the word `switch` followed by `Dallas, TX`, allowing flexible whitespace between terms. This signature is applied only to the first page of the OCR text during format detection.

The header configuration combines static fields and regex fields. Static fields are used when the value is known for this format and does not need to be extracted from OCR text:

```
"vendor_name": {
  "type": "static",
  "value": "Switch, Ltd."
},
"currency": {
  "type": "static",
  "value": "USD"
}
```

Regex fields are used when the value changes from invoice to invoice. For example, the invoice date, due date, and invoice number are extracted from a shared line containing three values:

```
"invoice_date": {
  "type": "regex",
  "pattern": "(\\d{2}/\\d{2}/\\d{2})\\s+\\d{2}/\\d{2}/\\d{2}\\s+\\d+",
  "group": 1
}
```

```
"due_date": {
  "type": "regex",
  "pattern": "\\d{2}/\\d{2}/\\d{2}\\s+(\\d{2}/\\d{2}/\\d{2})\\s+\\d+",
  "group": 1
}
```

```
"invoice_number": {
  "type": "regex",
  "pattern": "\\d{2}/\\d{2}/\\d{2}\\s+\\d{2}/\\d{2}/\\d{2}\\s+(\\d+)",
  "group": 1
}
```

The total amount is extracted from the text segment that starts with `TotalUSD`:

```
"total_amount": {
  "type": "regex",
  "pattern": "Total USD\\s*\\$\\s*([\\d,]+\\.\\d{2})",
  "group": 1
}
```

The bill-to name uses a broader expression because it must be located between the invoice-number block and the `AccountNo.` marker:

```
"bill_to_name": {
  "type": "regex",
```

```

"pattern":
  "\\d{2}/\\d{2}/\\d{2}\\s+\\d{2}/\\d{2}/\\d{2}\\s+\\d+\\s*\\n+([~\\n]+)\\n+(?:[~\\n]+\\n+)*?Account
  No\\. ",
"group": 1
}

```

The line-item table is configured with four main elements:

- **start_anchor**: identifies the beginning of the table.
- **stop_anchor**: identifies where table parsing must stop.
- **row_pattern**: extracts row values from each table line.
- **columns**: maps regex capture groups to output field names.

The table starts when the extractor finds the header line:

```
"start_anchor": "Description\\s+Quantity\\s+Rate\\s+Amount"
```

The table stops when the document reaches the total or footer text:

```
"stop_anchor": "^Total USD|Please update your system"
```

The row pattern is:

```

"row_pattern":
  "^(.*?)\\s+((?:-)?[\\d,]+\\.\\d+)\\s+((?:-)?[\\d,]+\\.\\d+)\\s+((?:-)?[\\d,]+\\.\\d+)$"

```

This pattern captures four groups:

1. description;
2. quantity;
3. rate;
4. amount.

The corresponding column mapping is:

```
"columns": ["description", "quantity", "rate", "amount"]
```

The first group uses a non-greedy text capture:

```
(.*?)
```

This allows the description to contain spaces while still letting the final three numeric groups be captured as quantity, rate, and amount. The numeric groups allow comma thousands separators and optional negative signs:

```
(?:-)?[\\d,]+\\.\\d+
```

This is why rows with discounts or negative amounts can still be parsed when the OCR text follows the configured pattern.

The current SKU logic extracts numeric identifiers from descriptions that match the Switch Fiber Pair pattern:

```

"sku": {
  "source_field": "description",
  "pattern": "(?i)^Transport\\s*\\\\\\\\s*Switch Fiber Pair\\b.*?\\bPairs\\s*\\\\\\\\(\\s*(\\d+)\\s*\\\\\\\\)",
  "group": 1
}

```

This means that SKU extraction is format-specific and configurable. The Python code does not know what a Switch SKU looks like. It only knows that, for this format, it must search the **description** field with the configured regex and return capture group 1.

For example, a description such as:

```
Transport | Switch Fiber Pair [Intra-campus] | Pairs (4419693704) ...
```

can produce:

```
"sku": "4419693704"
```

Rows whose descriptions do not match the configured SKU pattern receive:

```
"sku": null
```

This behavior is visible in the frontend table, where some rows have a resolved SKU and others show `None`. This is expected because the configured SKU rule is intentionally specific to Switch Fiber Pair descriptions.

Overall, the extraction methodology can be summarized as follows:

```
OCRtext → pagesplitting → formatdetection → headerextraction → anchor-basedtableddetection →
rowregexparsing → numericconversion → SKUresolution → structuredJSON
```

This approach is suitable for the current technical test because the target invoice layout is regular enough to be described through anchors and regular expressions. The main advantage is that the extraction logic can be adapted to new document types by adding configuration files, rather than rewriting the parser.

8 Validator Folder

The `components/validator/` folder implements the validation layer of the application. Its role is to verify whether the structured JSON produced by the extractor is internally consistent. The validator does not read PDF files, does not call `Veryfi`, does not perform OCR, and does not parse raw OCR text. It only receives the already extracted JSON object and applies rule-based consistency checks.

This component is important because the extraction process can produce plausible-looking data even when some OCR or parsing errors occurred. The validator acts as a quality-control layer between extraction and user review. It checks whether the numeric relationships inside the extracted invoice are coherent, for example whether each line item satisfies `quantity*rate=amount` and whether the sum of all line-item amounts matches the invoice total.

The validation methodology is configuration-driven. The Python code implements a generic rule engine, while the document-specific validation rules are stored in JSON files under:

```
components/validator/configs/
```

For the current technical test, the configured validation file is:

```
components/validator/configs/switch.json
```

This file defines the validation rules for the extracted `SwitchInvoice` format. The connection between extractor and validator is made through the `format` field in the extracted JSON. If the extractor returns:

```
{
  "format": "Switch Invoice",
  "header": {... },
  "pages": [... ]
}
```

then the validator searches for a validation configuration whose `format_name` is also `SwitchInvoice`.

8.1 Files and Responsibilities

File	Description
<code>components/validator/main.py</code>	Defines the FastAPI validator service. It creates the application, initializes a module-level <code>ValidatorEngine</code> , defines the request schema <code>ValidationRequest</code> , exposes <code>POST /validate</code> , logs validation requests, and returns the validation result produced by the engine.
<code>components/validator/core/rule_engine.py</code>	Implements the validation logic. It loads validation JSON files, stores them by <code>format_name</code> , selects the correct rule set for the incoming extracted document, consolidates line items from all pages, applies each configured rule, and returns a structured validation result.
<code>components/validator/configs/switch.json</code>	Defines the current validation rules for the <code>SwitchInvoice</code> format. It contains two rules: one row-level arithmetic check and one document-level sum check.
<code>components/validator/Dockerfile</code>	Defines the validator service container. It installs the validator-specific dependencies from <code>components/validator/requirements.txt</code> , copies the repository into <code>/app</code> , and starts the FastAPI service with Uvicorn on port 8003.
<code>components/validator/requirements.txt</code>	Lists the validator-specific dependencies: FastAPI, Uvicorn, and Pydantic.

The validator uses the shared logging utility:

```
logger = get_logger(__name__, "validator.log")
```

This means that configuration loading, validation requests, validation success, validation failures, unknown rule types, and numeric evaluation errors are written to the validator log file.

8.2 ValidatorEngine

The class `ValidatorEngine`, implemented in `components/validator/core/rule_engine.py`, is the core of the validation service. Its responsibility is to load validation configurations and apply them to extracted JSON documents.

When the validator service starts, `components/validator/main.py` creates a module-level engine:

```
engine = ValidatorEngine()
```

The constructor initializes an empty configuration dictionary and then calls `_load_configs()`:

```
self.configs: dict = {}
self._load_configs()
```

The configuration directory is resolved relative to the rule engine file:

```
config_dir = Path(__file__).parent.parent / "configs"
```

If the directory does not exist, it is created and no validation rules are loaded. If the directory exists, the engine reads every `*.json` file. Each valid file is stored using its `format_name`:

```
fmt_name = data.get("format_name")
if fmt_name:
    self.configs[fmt_name] = data
```

This means the validator can support multiple document types as long as each type has a corresponding JSON configuration with a unique `format_name`. Configuration files without `format_name` are skipped. Invalid or unreadable JSON files are logged and do not stop the service from loading other configurations.

The main public method is:

```
validate(extracted_data: dict) -> dict
```

The validation flow is:

1. Read the document format from `extracted_data["format"]`.
2. Search for a validation configuration with the same `format_name`.
3. If no configuration exists, return an invalid validation result.
4. Read the rule list from the selected configuration.
5. Consolidate all line items from all pages.
6. Read the header object.
7. Apply each configured validation rule.
8. Collect validation errors and detailed check results.
9. Return `is_valid`, `message`, `errors`, and `details`.

The line-item consolidation step is important. The extractor stores line items page by page:

```
"pages": [
  {
    "page_number": 1,
    "line_items": [... ]
  },
  {
    "page_number": 2,
    "line_items": [... ]
  }
]
```

The validator flattens this structure before applying rules:

```
all_items: list[dict] = []
for page in extracted_data.get("pages", []):
    all_items.extend(page.get("line_items", []))
```

Therefore, the validation rules are applied to the complete invoice, not only to a single page. If the extracted document has no known validation configuration, the engine returns:

```
{
  "is_valid": false,
  "message": "No validation config found for format '...'",
  "errors": []
}
```

Notice that in this specific branch the returned object does not include a `details` key. The normal validation path does include `details`. This is an implementation detail of the current code.

For known formats, the normal response structure is:

```
{
  "is_valid": true,
  "message": "Validated ",
  "errors": [],
  "details": [
    {
      "check": "Row 1 Math",

```

```

    "status": true,
    "message": "Row 1 valid: 2.0 * 50.0 == 100.0"
  },
  {
    "check": "Document Sum",
    "status": true,
    "message": "Sum valid: 125.0 == 125.0"
  }
]
}

```

If validation fails, `is_valid` becomes `false`, the `message` becomes `ValidationFailed`, and the `errors` list contains the failed checks.

The engine currently supports two rule types:

- `row_math`
- `document_sum`

Unknown rule types are not fatal. They are logged as warnings and skipped:

```
logger.warning("Unknown rule type '%s' skipping.", rule_type)
```

8.3 Current Validation Rules

The current validation configuration for Switch invoices is:

```
components/validator/configs/switch.json
```

It defines the following format name:

```
"format_name": "Switch Invoice"
```

This name must match the `format` field returned by the extractor. If the names differ, the validator will not find the configuration and will return an invalid result indicating that no validation config was found.

The Switch configuration currently defines two rules:

```

{
  "type": "row_math",
  "description": "Quantity * Rate = Amount",
  "expression": "quantity * rate",
  "equals": "amount",
  "tolerance": 0.1
}

```

and:

```

{
  "type": "document_sum",
  "description": "Sum of all amounts equals total_amount",
  "sum_column": "amount",
  "equals_header": "total_amount",
  "tolerance": 0.1
}

```

Although the `row_math` rule contains the fields `expression` and `equals`, the current Python implementation does not dynamically evaluate arbitrary expressions. Instead, `_apply_row_math()` is hard-coded to validate:

```
quantity * rate == amount
```

The rule's `tolerance` value is used to decide whether the difference between the expected value and the extracted amount is acceptable.

The row-level check is implemented in `_apply_row_math()`. For each line item, the validator reads:

```
q = float(str(row.get("quantity", 0)).replace(",", ""))
r = float(str(row.get("rate", 0)).replace(",", ""))
a = float(str(row.get("amount", 0)).replace(",", ""))
expected = q * r
```

This means that the validator accepts both numeric values and numeric strings. Commas are removed before conversion, so values such as "2,912.98" can be converted correctly.

If the absolute difference between `expected` and `amount` is greater than the configured tolerance, the row fails:

```
if abs(expected - a) > tolerance:
    ...
```

A failed row produces an error message such as:

```
Row 1 Math Error: Quantity (2.0) * Rate (50.0) != Amount (80.0)
```

and a detail entry:

```
{
  "check": "Row 1 Math",
  "status": false,
  "message": "Row 1 Math Error: Quantity (2.0) * Rate (50.0) != Amount (80.0)"
}
```

If the row is valid, the validator adds a successful detail entry and does not add anything to errors.

The document-level check is implemented in `_apply_document_sum()`. It reads the target column and target header from the rule:

```
sum_col = rule["sum_column"]
target_header = rule["equals_header"]
```

For the current Switch configuration, this means:

```
sum_col = "amount"
target_header = "total_amount"
```

The validator calculates the total sum from all line items:

```
total_calculated = sum(
    float(str(row.get(sum_col, 0)).replace(",", "")) for row in all_items
)
```

It then reads the expected invoice total from the extracted header:

```
target_val_str = header.get(target_header)
```

If the header field is missing or empty, validation fails with:

```
Header field 'total_amount' not found for sum check.
```

If the header field exists, it is converted to a float after removing commas:

```
target_val = float(str(target_val_str).replace(",", ""))
```

Then the calculated total and header total are compared using the configured tolerance:

```
if abs(total_calculated - target_val) > tolerance:
    ...
```

If the values differ by more than the tolerance, the validator produces an error such as:

```
Sum Error: Calculated sum of amount (100.0) != Header total_amount (125.0)
```

The validator is designed to fail safely. If numeric conversion fails, if a malformed value is found, or if an unexpected error occurs during rule evaluation, the service does not crash. Instead, it adds a validation error:

- `RowNMathEvaluationError` for row-level failures.
- `SumEvaluationError` for document-level sum failures.

The final result is determined only by the error list:

```
is_valid = len(errors) == 0
```

Therefore, a document is considered valid only when all configured checks complete without appending any errors.

From the full application perspective, the validator is called by the backend after extraction:

`OCRtext → ExtractorJSON → Validatorresult`

The backend then attaches the validator response to the extracted JSON under the `validation` key. This allows the frontend to show both the extracted data and the validation status to the user. The validator does not modify the extracted line items or header fields; it only reports whether they are consistent according to the configured rules.

9 Frontend Folder

The `frontend/` folder implements the user interface using Streamlit.

9.1 Files and Responsibilities

File	Description
<code>frontend/app.py</code>	Streamlit entry point. Sets page config, injects CSS, initializes session state, and routes between views.
<code>frontend/views/login.py</code>	Login and registration UI.
<code>frontend/views/dashboard.py</code>	Project dashboard with project CRUD actions.
<code>frontend/views/project.py</code>	Upload and results workspace for a selected project.
<code>frontend/utils/pdf_renderer.py</code>	Renders a selected PDF page as a PNG using PyMuPDF.
<code>frontend/utils/styles.py</code>	Injects custom CSS and displays the project logo.
<code>frontend/static/logo.png</code>	Logo used in the interface.
<code>frontend/Dockerfile</code>	Container definition for the Streamlit frontend.

9.2 Session State

The frontend uses `st.session_state` to store:

- `user`: authenticated user information.
- `current_project`: selected project ID.
- `pdf_bytes`: current PDF binary content.
- `pdf_page`: current page index.
- `extracted_data`: current extracted JSON object.

9.3 PDF Rendering

`frontend/utils/pdf_renderer.py` uses PyMuPDF to open PDF bytes directly from memory. The current page is rendered with a scaling matrix of 2.5x2.5, converted to PNG bytes, and displayed through `st.image`. Out-of-bounds pages and rendering exceptions are reported through `st.error` and logged.

10 Logs Folder and Centralized Logging

The application uses `logger.py` as a shared logging module. Every service obtains a logger through:

```
from logger import get_logger
logger = get_logger(__name__, "backend.log")
```

10.1 Logging Design

The logger writes to both console and rotating files:

Element	Behavior
Console handler	Logs INFO and above.
File handler	Logs DEBUG and above.
Rotation	Maximum 5 MB per log file.
Backups	Up to 5 rotated backups.
Default folder	<code>logs/</code> at project root.
Override	<code>LOG_DIR</code> environment variable.
Propagation	Disabled to avoid duplicate root logger output.

10.2 Service Log Files

The current implementation writes service-specific logs:

- `backend.log`;
- `ocr.log`;
- `extractor.log`;
- `validator.log`;
- `frontend.log`.

In Docker Compose, `./logs:/app/logs` is mounted into each service so logs remain visible on the host machine.

11 Tests Folder

The `tests/` folder contains a complete testing layer for the application. It is divided into unit tests, API-layer tests, fixtures, and report outputs.

11.1 Testing Structure

Listing 15: Testing folder structure

```

tests/
|-- api/
| |-- test_backend_api.py
| |-- test_extractor_api.py
| |-- test_ocr_api.py
| '-- test_validator_api.py
|-- unit/
| |-- test_backend_database.py
| |-- test_backend_services.py
| |-- test_document_parser.py
| |-- test_dynamic_format.py
| |-- test_extractor_file_handler.py
| |-- test_extractor_main.py
| |-- test_frontend_dashboard.py
| |-- test_frontend_utils.py
| |-- test_logger.py
| |-- test_ocr_config.py
| |-- test_ocr_file_utils.py
| |-- test_ocr_main.py
| |-- test_ocr_service.py
| |-- test_security.py
| '-- test_validator_engine.py
|-- fixtures/
|-- reports/
|-- conftest.py
|-- Dockerfile
'-- requirements.txt

```

11.2 Testing Philosophy

The suite separates true unit tests from API-layer tests. It deliberately avoids external dependencies:

- The real Verify API is never called.
- Real credentials are not required.
- Docker Compose is optional for tests.
- Temporary directories and temporary SQLite databases are used.
- External HTTP calls are mocked.
- Visual Streamlit layout is not tested as a unit test.

11.3 Unit Tests

The unit tests cover internal behavior directly:

Test file	Purpose
test_logger.py	Verifies logger handlers, registry behavior, log levels, and duplicate-handler prevention.
test_security.py	Verifies password hashing, bcrypt verification, wrong-password rejection, and plaintext fallback.
test_backend_services.py	Mocks HTTPX calls and verifies OCR, extractor, validator orchestration.
test_backend_database.py	Verifies database session lifecycle and rollback behavior.

<code>test_ocr_config.py</code>	Tests environment-based Verifyfi configuration loading.
<code>test_ocr_file_utils.py</code>	Tests hashing, cache read/write, OCR output paths, and file saving.
<code>test_ocr_service.py</code>	Mocks Verifyfi client behavior and tests <code>VerifyfiOCR</code> .
<code>test_ocr_main.py</code>	Tests CLI-level OCR flow with mocked filesystem/service behavior.
<code>test_dynamic_format.py</code>	Tests signature matching, header extraction, line-item parsing, continuation lines, numeric conversion, and SKU extraction.
<code>test_document_parser.py</code>	Tests page splitting, config loading, empty documents, unknown formats, and multi-page parsing.
<code>test_validator_engine.py</code>	Tests config loading, row math, document totals, numeric strings, invalid values, missing totals, and unknown rules.
<code>test_frontend_utils.py</code>	Tests PDF rendering and style/logo helper behavior using mocks.
<code>test_frontend_dashboard.py</code>	Tests the project-opening state transition logic.

11.4 API-Layer Tests

The API tests use FastAPI `TestClient` and monkeypatching:

Test file	Purpose
<code>test_backend_api.py</code>	Tests registration, login, project CRUD, PDF retrieval, and processing with mocked <code>run_pipeline</code> .
<code>test_ocr_api.py</code>	Tests cache hit, cache miss, uninitialized service, empty OCR text, unexpected OCR errors, and temp-file cleanup.
<code>test_extractor_api.py</code>	Tests successful extraction, parser-level errors, and unexpected extractor exceptions.
<code>test_validator_api.py</code>	Tests valid and invalid validation payloads.

11.5 Fixtures and Reports

The `tests/fixtures/` directory includes synthetic OCR text and extracted JSON examples. These fixtures represent valid and invalid Switch invoices, including invalid row math, invalid document sums, missing totals, and unknown formats.

The `tests/reports/` directory contains generated artifacts. The current included JUnit report indicates **86 tests collected, 86 passed, 0 failures, 0 errors, and 0 skipped**. The unit coverage XML indicates **91.74% line coverage** for the measured unit-test run.

11.6 Running the Tests

Install test dependencies:

```
python -m pip install -r tests/requirements.txt
```

Run all tests:

```
pytest -v
```


Run only unit tests:

```
pytest tests/unit -v
```

Run only API-layer tests:

```
pytest tests/api -v
```

Run the suite inside Docker:

```
docker compose --profile test run --rm tests
```

Generate coverage:

```
pytest -v --cov=. --cov-report=term-missing
```

12 Dockerization

12.1 Docker Compose Services

The root `docker-compose.yml` defines six services:

Service	Port/Profile	Description
ocr	8001	Builds from <code>components/ocr/Dockerfile</code> . Loads <code>.env</code> , mounts logs, and uses <code>ocr_cache</code> .
extractor	8002	Builds from <code>components/extractor/Dockerfile</code> . Provides OCR-text-to-JSON parsing.
validator	8003	Builds from <code>components/validator/Dockerfile</code> . Provides validation rules.
backend	8000	Builds from <code>backend/Dockerfile</code> . Depends on OCR, extractor, and validator. Uses <code>backend_data</code> .
frontend	8501	Builds from <code>frontend/Dockerfile</code> . Depends on backend.
tests	profile test	Builds from <code>tests/Dockerfile</code> . Runs pytest with <code>LOG_DIR=/tmp/test-logs</code> .

12.2 Volumes

Docker Compose defines two named volumes:

- `backend_data`: persists SQLite data and uploaded PDFs under `/app/data`.
- `ocr_cache`: persists OCR cache JSON files under `/app/ocr_cache`.

Additionally, `./logs:/app/logs` is bind-mounted into services so logs are accessible from the host.

12.3 Service Dockerfiles

Each service has a dedicated Dockerfile. They follow the same pattern: use a slim Python base image, set `/app` as working directory, install service-specific requirements, copy the repository, and start either Uvicorn or Streamlit.

The frontend image additionally installs `libmupdf-dev`, which supports PyMuPDF-based PDF rendering.

13 Configuration-Driven Generalization

13.1 Current Generalization Boundary

The main generalization mechanism is the extractor and validator configuration. The Python code does not hard-code the Switch invoice structure except through the active JSON files. A new document type can be added by creating:

- a new extractor config in `components/extractor/configs/`;
- a new validator config in `components/validator/configs/`;
- optional fixtures and tests for the new format.

13.2 Adding a New Extractor Format

A new extractor configuration should define:

```
{
  "format_name": "New Format Name",
  "signature_regex": "unique text that identifies the document",
  "header_fields": {
    "field_name": {
      "type": "regex",
      "pattern": "...",
      "group": 1
    }
  },
  "line_items": {
    "start_anchor": "...",
    "stop_anchor": "...",
    "row_pattern": "...",
    "columns": ["description", "quantity", "rate", "amount"],
    "sku": {
      "source_field": "description",
      "pattern": "...",
      "group": 1
    }
  }
}
```

The `signature_regex` should uniquely identify the document type from the first page. Header fields can be either static or regex-based. Line-item extraction requires a stable table header, a stop condition, a row regex, and a column mapping.

13.3 Adding a New Validator Format

A matching validator configuration should use the same `format_name`. For example:

```
{
  "format_name": "New Format Name",
  "rules": [
    {
      "type": "row_math",
      "description": "Quantity * Rate = Amount",
      "tolerance": 0.1
    },
    {
      "type": "document_sum",
      "sum_column": "amount",
      "equals_header": "total_amount",
      "tolerance": 0.1
    }
  ]
}
```

```
]
}
```

13.4 Recommended Future Improvements

1. **Persist manual corrections:** add backend endpoints to save edited line items after the user modifies them in the frontend.
2. **Format selection audit:** store the selected parser format and config version used for each processed document.
3. **Config schema validation:** define Pydantic models or JSON Schema files for extractor and validator configs.
4. **Multiple OCR providers:** abstract the OCR layer so Veryfi can be replaced or complemented by another provider.
5. **Structured validation rules:** add rule types such as required fields, date consistency, duplicate SKU detection, currency validation, and per-column bounds.
6. **Database migration layer:** use Alembic instead of automatic table creation for production deployments.
7. **Authentication hardening:** replace simple login responses with token-based sessions if exposed outside a trusted environment.
8. **Role-based access control:** add user roles if multiple users or reviewers need different permissions.
9. **Frontend persistence model:** separate display-only edits from committed corrections.
10. **CI/CD integration:** run `pytest`, coverage, linting, and Docker build checks automatically on every pull request.

14 Operational Notes and Known Constraints

- The backend stores extracted JSON as text in SQLite. This is simple and suitable for the technical test, but a normalized schema may be more appropriate for analytical workloads.
- The current frontend table editor modifies in-memory session data only. A save endpoint is needed for persistent manual review workflows.
- OCR depends on Veryfi credentials and external API availability unless the result is already cached.
- Extraction quality depends strongly on OCR text stability and regular expression robustness.
- The current document format is Switch Invoice. Other formats require additional JSON configurations.
- The testing suite mocks external providers and HTTP boundaries. It is not an end-to-end production deployment test.

15 Summary

The Smart Invoice Extractor is structured as a modular Python system with clear service boundaries. The backend orchestrates the workflow, OCR is isolated behind a Veryfi adapter and content-hash cache, extraction is driven by JSON format definitions, validation is rule-based, the frontend provides user-facing project and review workflows, and the test suite verifies both internal logic and API-layer behavior without depending on external services.

This design is appropriate for a technical test because it demonstrates separation of concerns, containerization, logging, configurable parsing, validation logic, UI integration, persistence, and automated testing. The most important next step for production-like use would be to persist user corrections, formalize configuration schemas, and expand the format/rule system for additional document families.